

Buenas Prácticas y Seguridad en Contenedores Web

Pensar como atacante, programar como
defensor

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Lunes 17 de agosto de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Identificar riesgos comunes de seguridad en contenedores web (IIS y Node.js/Express).
2. Aplicar medidas básicas de protección con código TypeScript y configuración del sistema operativo.
3. Justificar técnicamente por qué cada medida reduce el riesgo – no memorizar una lista, sino entenderla.



Conexión con la Clase 8

Recordatorio: en la Clase 8 vimos que una ruta que confía ciegamente en el parámetro del usuario (monto) puede comportarse mal. Esa fue nuestra primera probada de seguridad.

Hoy: antes del laboratorio integrador, sistematizamos cómo no dejar el contenedor expuesto a riesgos básicos – pensando como atacante, no solo como desarrollador.

Esta es la última clase teórica de la Unidad 1 – el Parcial 1 y el Laboratorio 1 vienen pronto.



La mentalidad correcta: pensar como atacante

Una pregunta que cambia cómo programan para siempre:

"Si yo quisiera romper esto, ¿por dónde entraría?"

No es paranoia – es el ejercicio mental básico de cualquier ingeniero de sistemas serio.

Analogía

Cada ruta que acepta parámetros del usuario es una **puerta** de su edificio. Su trabajo no es solo construir la puerta – es decidir **qué puede pasar por esa puerta y qué no**, y qué hacer con quien intenta forzarla.



Medida 1: validar siempre la entrada del usuario

Ya lo vimos en la Clase 8 – hoy lo formalizamos con límites más completos:

```
1 app.get("/cotizacion", (req: Request, res: Response) => {
2   const montoRaw = req.query.monto as string;
3   const monto = Number(montoRaw);
4
5   if (!montoRaw || isNaN(monto) || monto < 0 || monto > 1_000_000) {
6     res.status(400).json({ error: "Monto invalido" });
7     return;
8   }
9   // continuar con la logica solo si el dato es valido
10 });
```

Nuevo respecto a la Clase 8: el límite superior ($> 1_000_000$). Sin tope máximo, alguien podría enviar un número absurdamente grande para forzar errores o comportamientos inesperados en cálculos posteriores.



Medida 2: no exponer detalles internos en errores

```
1 // MAL -- expone informacion interna al usuario
2 app.use((err: Error, req: Request, res: Response, next: Function) => {
3   res.status(500).send(err.stack); // nunca hacer esto en produccion
4 });
5
6 // BIEN -- mensaje generico al usuario, detalle solo en el log
7 app.use((err: Error, req: Request, res: Response, next: Function) => {
8   console.error(err); // esto si queda en el log, que solo ve el administrador
9   res.status(500).json({ error: "Ocurrio un error interno" });
10  });
```

Mismo error, dos destinos distintos: el detalle técnico se queda en el log (que solo ustedes ven), y el usuario recibe un mensaje que no le sirve a un atacante.



¿Qué revela exactamente un `err.stack`?

Lo que un atacante puede leer ahí

- Rutas absolutas del servidor (`/opt/miapp/servidor.js`, línea exacta del error).
- Nombres de librerías y versiones instaladas – útil para buscar vulnerabilidades conocidas de esa versión.
- A veces, fragmentos de la lógica interna o nombres de variables.

*Nada de esto es "secreto" en el sentido de contraseñas – pero es **información gratis** que le ahorra trabajo a quien está buscando cómo entrar. Entre menos sepa el atacante, más tiempo y esfuerzo necesita.*



Medida 3: cabeceras de seguridad con helmet

Una librería que aplica buenas prácticas automáticamente:

```
1 npm install helmet
```

```
1 import helmet from "helmet";  
2 app.use(helmet());
```

¿Qué es una "cabecera HTTP de seguridad"?

Información adicional que el servidor manda junto con la respuesta, indicándole al navegador **cómo comportarse** con ese contenido (ej. "no permitas que este sitio se muestre dentro de otro en un iframe"). `helmet` agrega varias de estas de un solo golpe, sin que ustedes memoricen cada una.



Medida 4: no correr el proceso como root

Ya lo aplicamos en la Clase 7 con el usuario `appweb` – hoy lo reconectamos como parte de una lista completa de seguridad, no como un paso aislado.

Recordatorio rápido: si la aplicación corre con privilegios mínimos, una falla de seguridad explotada limita el daño a lo que ese usuario puede hacer – no a todo el sistema.

Esto es exactamente el mismo principio de menor privilegio que usamos al crear `appweb` – hoy van a ver que se repite en todas las demás medidas.



Medida 5: limitar peticiones por usuario

Protección básica contra abuso:

```
1 npm install express-rate-limit
```

```
1 import rateLimit from "express-rate-limit";  
2  
3 const limitador = rateLimit({ windowMs: 60_000, max: 100 });  
4 app.use(limitador);
```

¿Por qué importa?

Sin límite, alguien (o un programa automatizado) podría mandar miles de peticiones por segundo – saturando el servidor para usuarios legítimos, o probando miles de combinaciones de datos en segundos (ej. adivinar contraseñas). `windowMs: 60_000`, `max: 100` significa **máximo 100 peticiones por minuto, por usuario**.



Seguridad en IIS

Medida	Cómo aplicarla	Por qué
Deshabilitar <i>directory browsing</i>	IIS Manager → Directory Browsing → Disable	Evita exponer listados de archivos del servidor
Application Pool por sitio (Clase 6)	Aislamiento de procesos	Un sitio comprometido no afecta a otros
Ocultar errores detallados	<code>web.config</code> → <code>customErrors</code>	No exponer rutas internas ni mensajes técnicos
Usar HTTPS (certificado TLS)	Bindings → agregar https	Cifra el tráfico, evita interceptación de datos
Mantener actualizado	Windows Update	Corrige vulnerabilidades conocidas del sistema



Principio transversal: menor privilegio

Tanto en Express/Node como en IIS, la idea central es la misma:

Cada componente debe tener solo el acceso mínimo necesario para funcionar, nunca más.

En Express/Node: usuario appweb sin privilegios (Clase 7), validación de entrada, errores sin detalle.

En IIS: Application Pool aislado por sitio (Clase 6), errores ocultos, HTTPS.

Cinco medidas distintas hoy, pero todas son la misma idea aplicada en un lugar diferente.



Ejercicio individual

Sobre el contenedor Express de las Clases 7-8, aplicar y documentar:

1. Instalar y activar `helmet`.
2. Instalar y activar `express-rate-limit` con un límite de **20 peticiones por minuto**.
3. Agregar el middleware de manejo de errores que **no** expone el detalle interno al usuario.
4. Probar haciendo más de 20 peticiones rápidas a una ruta (recargar repetidamente) y confirmar que el límite se activa.

Entregar: código modificado + captura del mensaje de "demasiadas peticiones" + breve justificación de cada medida (1-2 líneas).



Pista para el ejercicio

Pista

`rateLimit({ windowMs: 60_000, max: 100 })` del ejemplo de la teoría usa **100** – para el ejercicio solo cambien ese número a **20**. El resto del código es idéntico.

Para probar el límite más rápido, recarguen la página muy seguido (F5 repetidamente) o abran la misma URL en varias pestañas a la vez – 20 peticiones se alcanzan rápido.



Verificación de aprendizaje

1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?





Verificación de aprendizaje

1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?
Expone información interna del servidor que un atacante podría usar.





Verificación de aprendizaje

1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?
Expone información interna del servidor que un atacante podría usar.
2. ¿Qué hace la librería `helmet`?





Verificación de aprendizaje

1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?
Expone información interna del servidor que un atacante podría usar.
2. ¿Qué hace la librería `helmet`?
Aplica automáticamente varias cabeceras HTTP de seguridad recomendadas.





Verificación de aprendizaje

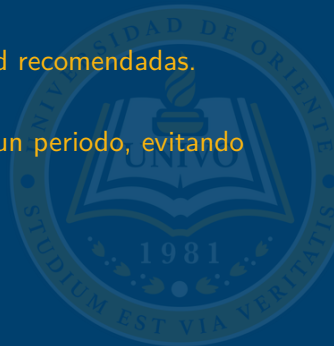
1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?
Expone información interna del servidor que un atacante podría usar.
2. ¿Qué hace la librería `helmet`?
Aplica automáticamente varias cabeceras HTTP de seguridad recomendadas.
3. ¿Qué problema resuelve `express-rate-limit`?





Verificación de aprendizaje

1. ¿Por qué no se debe enviar `err.stack` directamente al usuario?
Expone información interna del servidor que un atacante podría usar.
2. ¿Qué hace la librería `helmet`?
Aplica automáticamente varias cabeceras HTTP de seguridad recomendadas.
3. ¿Qué problema resuelve `express-rate-limit`?
Limita cuántas peticiones puede hacer un mismo usuario en un periodo, evitando abuso.





Resumen de la clase

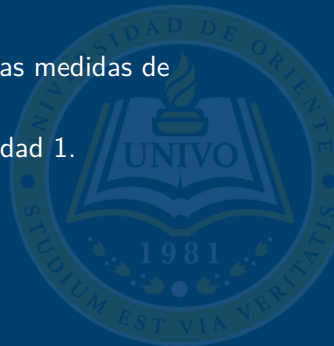
Contenedor	Medida prioritaria
Express/TypeScript	Validar entrada + helmet + rate limiting + no exponer errores
IIS	Deshabilitar directory browsing + HTTPS
Ambos	Menor privilegio, logs, actualizaciones



Preparación para la Clase 10

Tema: Laboratorio guiado – despliegue integrador (repaso antes del Parcial 1 y Laboratorio 1).

- Traer ambos contenedores (IIS y Express) funcionando, con las medidas de seguridad de hoy aplicadas.
- Repasar las Clases 2 a 9 – el Laboratorio 1 cubre toda la Unidad 1.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

